

```
@file:Suppress("DEPRECATION")
```

```
@file:OptIn(ExperimentalMaterial3Api::class)
```

Suppresses warnings for deprecated ArcGIS APIs (like layers)

Allows using experimental Material3 components

```
kotlin
```

Copy code

```
import androidx.compose.foundation.layout.*
```

```
import androidx.compose.material3.*
```

```
import androidx.compose.runtime.*
```

```
import androidx.compose.ui.Modifier
```

```
import androidx.compose.ui.unit.dp
```

```
import androidx.compose.ui.platform.LocalContext
```

Import Jetpack Compose UI tools

Used for layouts, buttons, text, state, padding, etc.

```
kotlin
```

Copy code

```
import com.arcgismaps.ArcGISEnvironment
```

```
import com.arcgismaps.data.*
```

```
import com.arcgismaps.geometry.*
```

```
import com.arcgismaps.mapping.*
```

```
import com.arcgismaps.mapping.layers.*
```

```
import com.arcgismaps.mapping.view.geometryeditor.GeometryEditor
```

```
import com.arcgismaps.toolkit.geoviewcompose.MapView
```

Import ArcGIS SDK classes:

Maps

Geometry

Feature tables

Layers

Geometry editor

Compose MapView

kotlin

Copy code

```
import kotlinx.coroutines.launch
```

Used to run background work (adding features to tables)

kotlin

Copy code

```
@Composable
```

```
fun GeoTiffMapScreen() {
```

Main UI function for your map screen

kotlin

Copy code

```
val context = LocalContext.current
```

```
ArcGISEnvironment.applicationContext = context.applicationContext
```

Gives ArcGIS SDK the Android application context

Mandatory for ArcGIS to work

kotlin

Copy code

```
val geometryEditor = remember { GeometryEditor() }
```

Creates a geometry editor

Used to draw point, line, and polygon

kotlin

Copy code

```
val scope = rememberCoroutineScope()
```

Coroutine scope to run background tasks safely

Feature Tables (data storage)

kotlin

Copy code

```
val pointTable = remember { ... }
```

Table to store point features

kotlin

Copy code

```
val lineTable = remember { ... }
```

Table to store line (polyline) features

kotlin

Copy code

```
val polygonTable = remember { ... }
```

Table to store polygon features

Each table:

kotlin

Copy code

```
FeatureCollectionTable(  
    listOf(Field(FieldType.Text, "name", "Name", 50)),  
    GeometryType.Point,  
    SpatialReference.wgs84()  
)
```

Means:

One attribute field named "name"

Geometry type = Point / Polyline / Polygon

Spatial reference = WGS84 (lat-long)

Feature Collection Layer

kotlin

Copy code

```
val featureCollectionLayer = remember {  
    FeatureCollectionLayer(  
        FeatureCollection().apply {  
            tables.add(pointTable)        }  
    )  
}
```

```

        tables.add(lineTable)

        tables.add(polygonTable)
    }
)
}

```

Groups all three tables into one layer

Internally creates:

A FeatureLayer for points

A FeatureLayer for lines

A FeatureLayer for polygons

Map creation

kotlin

Copy code

```

val map = remember {
    ArcGISMap(BasemapStyle.ArcGISLightGray).apply {
        operationalLayers.add(featureCollectionLayer)
    }
}

```

Creates a map with a light gray basemap

Adds your FeatureCollectionLayer to draw features

kotlin

Copy code

```

initialViewpoint = Viewpoint(...)

```

Sets the starting camera position and zoom level

UI State

kotlin

Copy code

```
var showPoints by remember { mutableStateOf(true) }  
var showLines by remember { mutableStateOf(true) }  
var showPolygons by remember { mutableStateOf(true) }
```

Controls visibility of each geometry type

Connected to checkboxes

Layer visibility logic

kotlin

Copy code

```
LaunchedEffect(showPoints, showLines, showPolygons) {
```

Runs whenever any checkbox value changes

kotlin

Copy code

```
val layers = featureCollectionLayer.layers.toList()
```

Converts ArcGIS internal layer list to Kotlin list

kotlin

Copy code

```
val pointLayer = layers[0] as Layer
```

First layer = point layer

kotlin

Copy code

```
pointLayer.isVisible = showPoints
```

Shows or hides point layer based on checkbox

Same for line and polygon layers

MapView

kotlin

Copy code

```
MapView(  
    arcGISMap = map,  
    geometryEditor = geometryEditor  
)
```

Displays the map

Connects geometry editor to allow drawing

Visibility UI

kotlin

Copy code

```
Checkbox(checked = showPoints, onCheckedChange = { showPoints = it })
```

Checkbox for point visibility

Same for line and polygon

Drawing buttons

kotlin

Copy code

```
geometryEditor.start(GeometryType.Point)
```

Start drawing points

kotlin

Copy code

```
geometryEditor.start(GeometryType.Polyline)
```

Start drawing lines

kotlin

Copy code

```
geometryEditor.start(GeometryType.Polygon)
```

Start drawing polygons

Save Geometry Button

kotlin

Copy code

```
val geometry = geometryEditor.stop()
```

Stops drawing and gets drawn shape

kotlin

Copy code

```
when (it) {  
    is Point -> add to pointTable  
    is Polyline -> add to lineTable  
    is Polygon -> add to polygonTable  
}
```

Decides which table to save into

Inside each:

kotlin

Copy code

```
val feature = table.createFeature()  
feature.geometry = it  
feature.attributes.set("name", "Point")  
table.addFeature(feature)
```

Means:

Create empty feature

Set its geometry

Set its attribute

Save it into the table

Cancel Button

kotlin

Copy code

```
geometryEditor.clearGeometry()
```

Removes current drawing

Cancels editing

In simple words:

This code builds an offline map drawing app where:

You can draw:

Points

Lines

Polygons

You can:

Save drawings

Show/hide each type separately

Cancel drawing

Everything:

Runs offline

Uses memory tables

Uses modern ArcGIS 100.x SDK

Uses Jetpack Compose UI